```
*/
extern void __send_remote_softirq(struct call_single_data *cp, int cpu,
                  int this_cpu, int softirq);
```

After their introduction the only issue was the compatibility with existing drivers.

softirqs are essentially a set of 32 statically defined bottom halves. They can be made to run in two similar processors, simultaneously. ***tasklets***, unlike ***softirqs***, are dynamic and are actually built on top of ***softirqs***. And they can be made to run on different processors, in parallel.

When you begin actual programming, you can see that ***tasklets*** are enough for handling your bottom half processing requirements. You may also note that sometimes we need ***softirqs*** for tasks like networking (owing to reasons concerning performance). The only point you need to keep in your mind is that this requires much attention. Now let's have a glance at the initiation:

```
#ifndef __ARCH_IRQ_STAT
irq_cpustat_t irq_stat[NR_CPUS] ____cacheline_aligned;
EXPORT_SYMBOL(irq_stat);
#endif

static struct softirq_action softirq_vec[32] __cacheline_aligned_in_smp;
static DEFINE_PER_CPU(struct task_struct *, ksoftirqd);

static inline void wakeup_softirqd(void)
{
    /* Interrupts are disabled: no need to stop preemption */
    struct task_struct *tsk = __get_cpu_var(ksoftirqd);

    if (tsk && tsk->state != TASK_RUNNING)
        wake_up_process(tsk);
}

#define MAX_SOFTIRQ_RESTART 10

asmlinkage void __do_softirq(void)
{
    struct softirq_action *h;
    __u32 pending;
    int max_restart = MAX_SOFTIRQ_RESTART;
    int cpu;

    pending = local_softirq_pending();

    local_bh_disable();
    cpu = smp_processor_id();
restart:
    /* Reset the pending bitmask before enabling irqs */
    local_softirq_pending() = 0;

    local_irq_enable();

    h = softirq_vec;

    do {
        if (pending & 1) {
            h->action(h);
            rcu_bh_qsctr_inc(cpu);
        }
        h++;
        pending >>= 1;
    } while (pending);

    local_irq_disable();

    pending = local_softirq_pending();
    if (pending && --max_restart)
        goto restart;

    if (pending)
        wakeup_softirqd();

    __local_bh_enable();
}
#ifndef __ARCH_HAS_DO_SOFTIRQ

asmlinkage void do_softirq(void)
{
    __u32 pending;
    unsigned long flags;

    if (in_interrupt())
        return;

    local_irq_save(flags);

    pending = local_softirq_pending();

    if (pending)
        __do_softirq();

    local_irq_restore(flags);
}

EXPORT_SYMBOL(do_softirq);

#endif
```

Another aspect that needs the attention of the programmer is that ***softirqs*** need to be registered statically (during compilation) while the code can dynamically register ***tasklets***. You may also note that converting BHs to ***softirqs*** (or even tasklets) is a non-trivial thing!

Fortunately, the 'conversion' later materialised in the 2.5 series development. ***tasklet*** finally appeared in the apparel of a modified ***softirq***, which could be handled easily. (Now you can understand why some authors of ***literature-type-texts*** refer bottom halves as software interrupts or ***softirqs***.) This finally led to